

STUDENT PERFORMANCE ANALYZER

Project Report

CSE1021 – Introduction to Problem Solving and Programming

Submitted By

Saurabh

B.Tech – Computer Science and Engineering

VITyarthi Project

VIT University

2026

1. Introduction

The Student Performance Analyzer is a Python-based application developed to store, manage, and analyze the academic performance of students. The system allows users to maintain student records, enter and update marks for different subjects, and generate meaningful performance information from the entered data.

The project is designed for students and teachers who need a simple way to view and analyze academic records. Instead of manually calculating totals, averages, grades, and results, the system performs these operations automatically.

The application is implemented using Python and follows a modular structure. Different tasks such as student management, marks management, performance analysis, class analysis, and report generation are organized into separate Python modules. This makes the program easier to understand, test, maintain, and extend.

The project applies concepts covered in CSE1021 – Introduction to Problem Solving and Programming, including variables, conditional statements, loops, functions, modules, input/output operations, data validation, and basic problem-solving techniques.

2. Problem Statement

Students often have their academic marks stored or viewed individually, which makes it difficult to analyze overall performance in a structured manner.

Calculating totals, averages, grades, and pass or fail results manually can also take additional time and may lead to errors.

The Student Performance Analyzer addresses this problem by providing a structured Python-based system for managing student records and analyzing academic marks. The system allows users to store student details, enter and update marks, calculate performance measures, search for student records, and generate individual and class-level analysis.

The system is intended to provide a simple and organized approach to academic performance analysis for students and teachers.

2.1 Objectives

The main objectives of the Student Performance Analyzer are:

- To maintain student academic records in an organized manner.
- To allow users to enter and update marks for different subjects.
- To automatically calculate total marks and average marks.
- To determine grades and pass/fail results based on the calculated average.
- To provide individual student performance analysis.
- To provide class-level performance analysis.
- To generate a structured student performance report.
- To apply Python programming concepts through a modular project implementation.

3. Functional Requirements

The Student Performance Analyzer provides the following functional requirements:

3.1 Add Student

The system shall allow the user to add a new student by entering a unique Student ID and student name.

3.2 View Student Records

The system shall allow the user to view the list of students currently stored in the system.

3.3 Search Student

The system shall allow the user to search for a student using the Student ID.

3.4 Update Student

The system shall allow the user to update the stored student information.

3.5 Enter and Update Marks

The system shall allow the user to enter or update marks for CSE, Calculus, English, and EVS. Marks must be within the valid range of 0 to 100.

3.6 Analyze Student Performance

The system shall calculate the student's total marks and average, determine the grade, and display the pass/fail result.

3.7 Class Analysis

The system shall provide class-level analysis including class average, highest and lowest performers, and grade distribution.

3.8 Generate Student Report

The system shall generate a structured performance report containing the student's subject marks, total, average, grade, and result.

4. Non-Functional Requirements

The following non-functional requirements define the quality and operational characteristics of the Student Performance Analyzer.

4.1 Usability

The system should provide a simple menu-driven interface so that users can easily access different operations such as adding students, entering marks, searching records, and generating reports.

4.2 Accuracy

The system should correctly calculate total marks, average marks, grades, pass/fail results, and class-level statistics based on the entered data.

4.3 Reliability

The system should handle invalid inputs and missing student records without terminating unexpectedly.

4.4 Performance

The system should perform student searches, calculations, and analysis efficiently for the intended class-sized dataset.

4.5 Maintainability

The system should use separate Python modules for different responsibilities so that individual parts of the application can be modified or extended without unnecessarily affecting other parts.

5. System Architecture

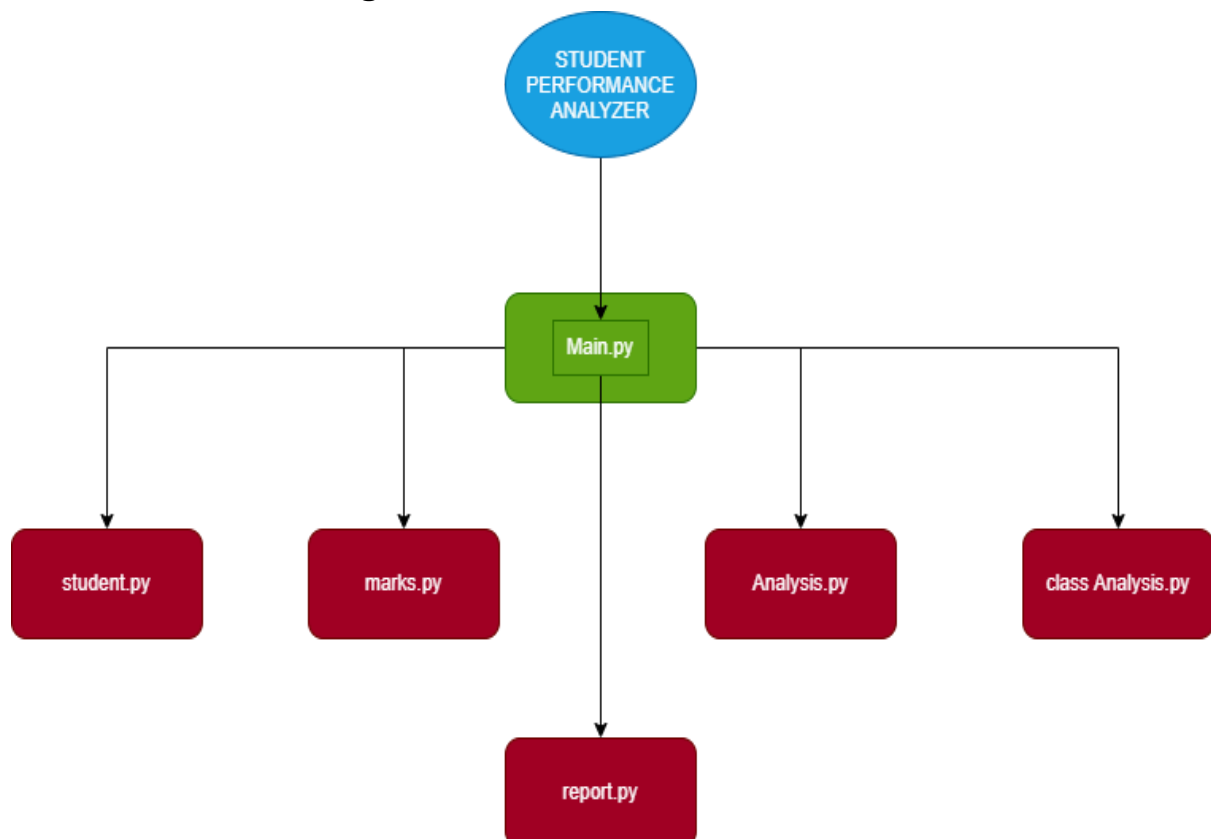
The Student Performance Analyzer follows a modular architecture in which the main program coordinates different Python modules, with each module responsible for a specific part of the system.

The main program, `main.py`, provides the menu-driven interface and controls the overall flow of the application. It connects with separate modules for student management, marks management, performance analysis, class analysis, and report generation.

The major modules used in the project are:

- **student.py** – Handles adding, viewing, searching, and updating student records.
- **marks.py** – Handles entering and updating subject marks and validates marks within the range of 0 to 100.
- **Analysis.py** – Calculates total marks, average marks, grades, and pass/fail results.
- **Class Analysis.py** – Performs class-level analysis such as class average, highest and lowest performers, and grade distribution.
- **report.py** – Generates an individual student's performance report.

5.1 Architecture Diagram



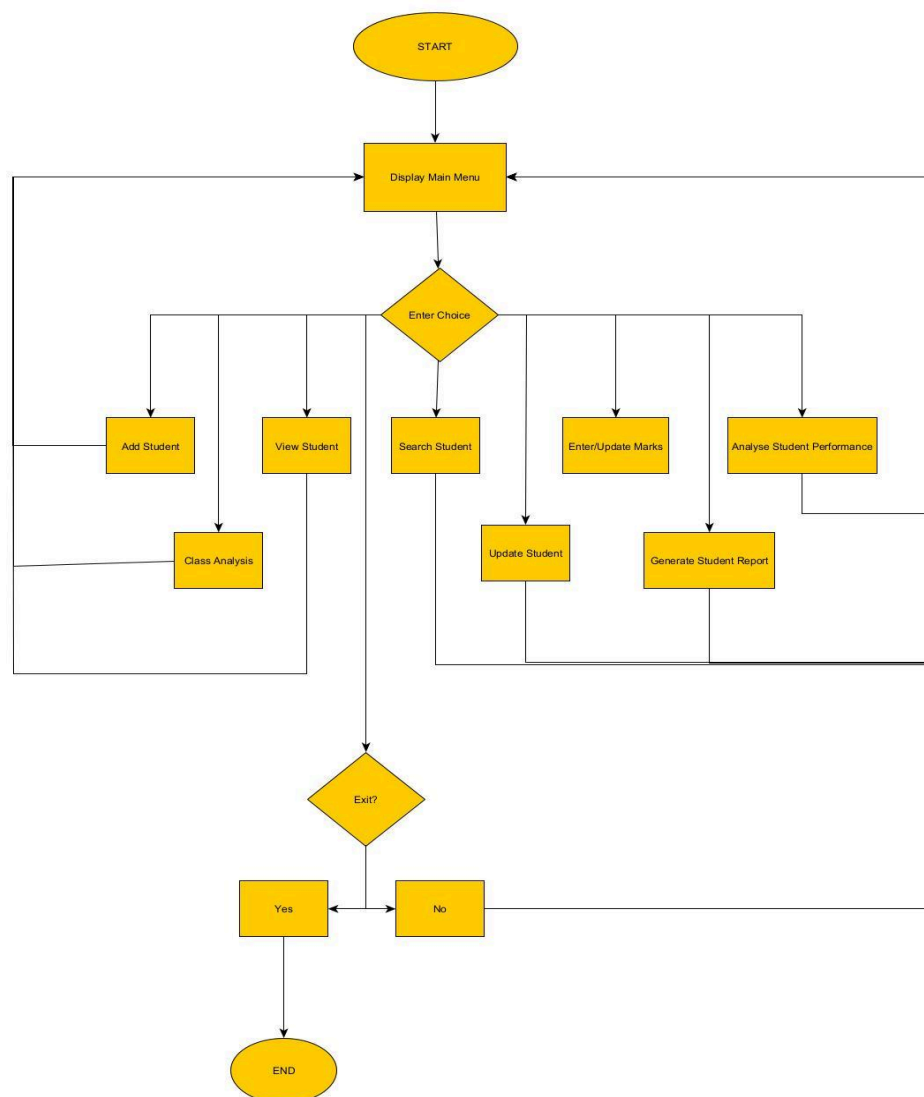
6. Workflow

The workflow of the Student Performance Analyzer describes how the user interacts with the system from starting the application to performing different operations.

When the program starts, the main menu is displayed. The user selects an operation by entering a menu choice. Depending on the selected option, the system performs the corresponding operation such as adding a student, viewing records, searching for a student, updating student information, entering marks, analyzing performance, performing class analysis, or generating a report.

After completing an operation, the system returns to the main menu, allowing the user to perform another operation. The program terminates when the user selects the exit option.

6.1 Workflow Diagram



7. UML Module Diagram

The UML module diagram represents the modular structure of the Student Performance Analyzer and shows how the main program is connected to the different Python modules used in the project.

The system is organized into separate modules, with each module handling a specific responsibility. The `main.py` module acts as the central controller and connects the student management, marks management, performance analysis, class analysis, and report generation modules.

7.1 Module Responsibilities

student.py

Handles student-related operations including adding students, viewing student records, searching for students, and updating student information.

marks.py

Handles entering and updating marks for the four subjects and validates that marks are within the range of 0 to 100.

Analysis.py

Performs individual performance calculations including total marks, average marks, grade calculation, and pass/fail determination.

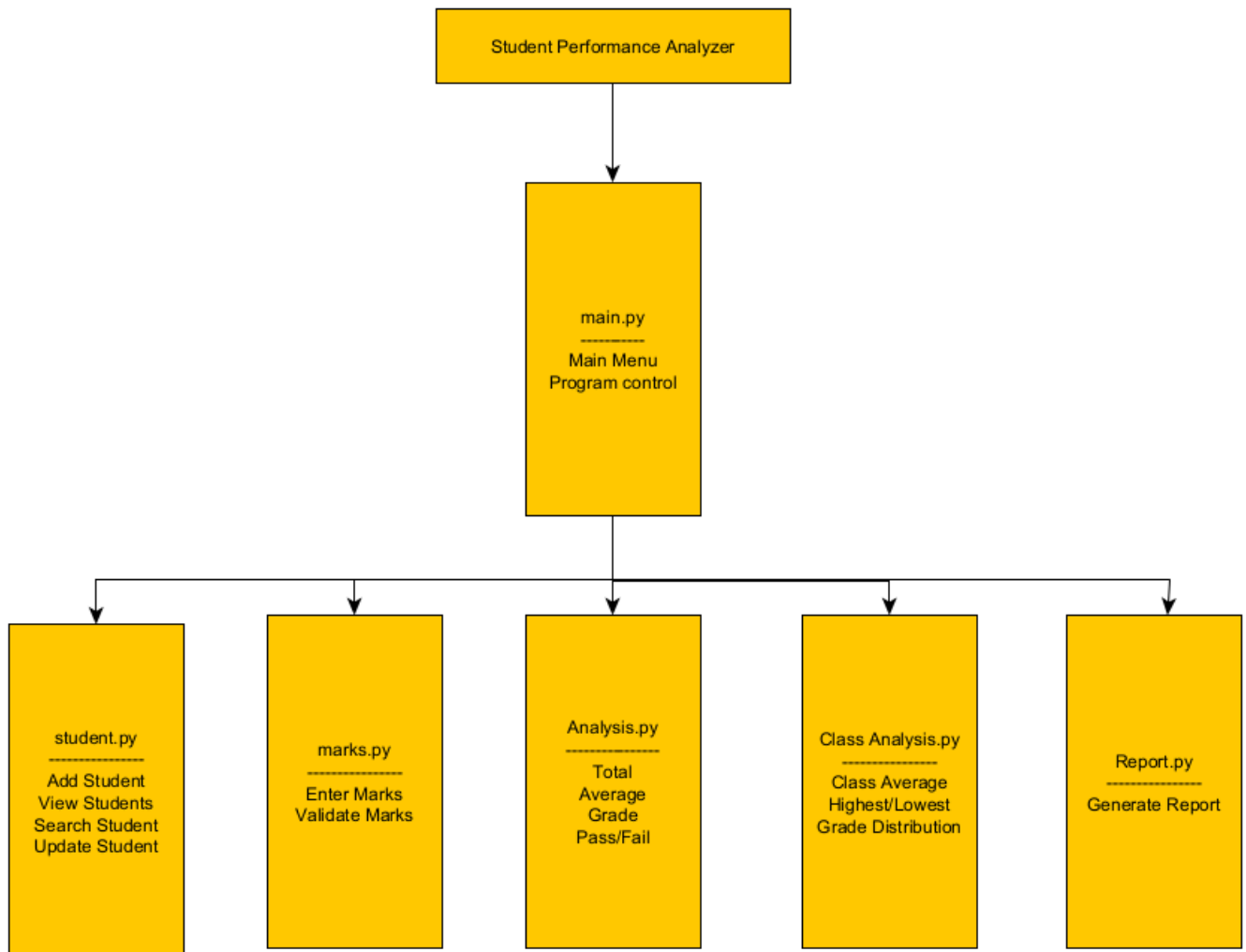
Class Analysis.py

Performs class-level analysis including class average, highest and lowest performers, and grade distribution.

Report.py

Generates a structured performance report for an individual student.

7.2 UML Module Diagram



8. Design Decisions

The following design decisions were made during the development of the Student Performance Analyzer:

8.1 Fixed Subject Structure

The system uses four predefined subjects: CSE, Calculus, English, and EVS. A fixed subject structure keeps the input format consistent for all students and simplifies performance calculations.

8.2 Separate Student and Marks Operations

Student information and marks entry are handled as separate operations. This allows student records to be created first and marks to be entered or updated later.

8.3 Modular Program Structure

The program is divided into separate Python modules based on functionality. Student management, marks management, performance analysis, class analysis, and report generation are handled separately. This improves organization and maintainability.

8.4 Input Validation

The system validates student IDs and marks entered by the user. Marks are accepted only when they are within the range of 0 to 100, and invalid numeric input is handled using error checking.

8.5 Grading and Result Logic

The system calculates the average marks and uses predefined grade ranges to determine the student's grade. A student with an average of 40 or above is considered to have passed.

9. Implementation

The Student Performance Analyzer was implemented using Python as a menu-driven console application. The implementation uses functions, conditional statements, loops, input/output operations, modules, dictionaries, lists, and input validation.

The program is divided into multiple modules according to their responsibilities. The `main.py` file controls the overall program flow and displays the main menu. Other modules perform specific operations related to student records, marks, performance analysis, class analysis, and report generation.

9.1 Student Management

The student management module handles the creation and management of student records. It provides functions to add a student, view stored students, search for a student using the Student ID, and update student information. Duplicate Student IDs are prevented during student creation.

9.2 Marks Management

The marks management module handles the entry and updating of marks for CSE, Calculus, English, and EVS. The system validates the entered marks and accepts values only between 0 and 100. Invalid numeric input is handled using error checking.

9.3 Performance Analysis

The performance analysis module calculates the total marks and average marks of a student. It then determines the grade and pass/fail result using the defined grading and result rules. The module also displays the marks obtained in individual subjects.

9.4 Class Analysis

The class analysis module processes the available student records to calculate the class average, identify the highest and lowest performers, and display the distribution of grades.

9.5 Report Generation

The report module generates an individual student performance report containing subject-wise marks, total marks, average marks, grade, and pass/fail result.

9.6 Course Concepts Applied

The project applies concepts from CSE1021 including:

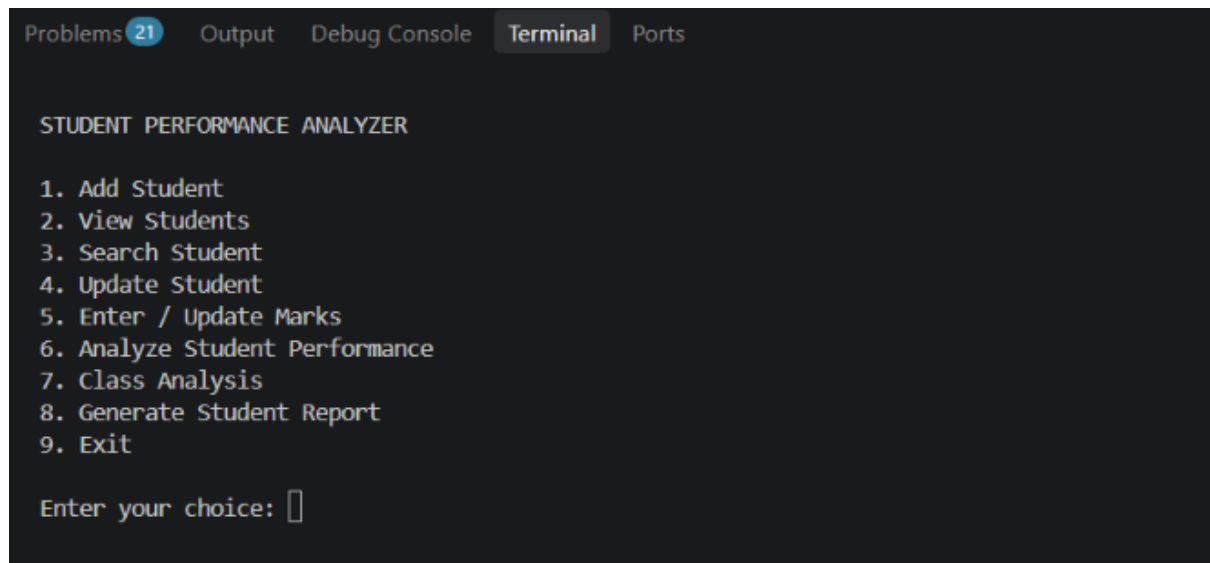
- Variables and data types
- Input and output operations
- Conditional statements
- Loops
- Functions
- Lists and dictionaries
- User-defined modules
- Input validation and error handling
- Basic problem-solving and algorithmic thinking

10. Screenshots and Results

The following screenshots demonstrate the working of the Student Performance Analyzer and show the outputs produced by the system for different operations.

10.1 Main Menu

The main menu provides access to all major operations of the Student Performance Analyzer.



```
Problems 21 Output Debug Console Terminal Ports

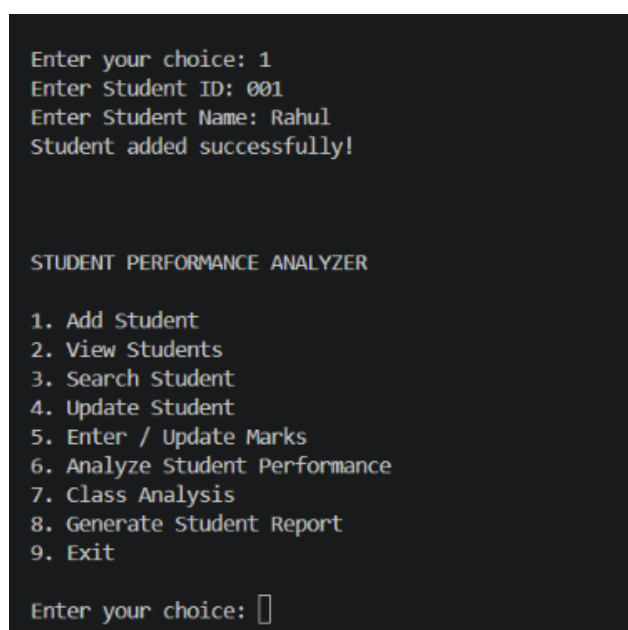
STUDENT PERFORMANCE ANALYZER

1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Enter / Update Marks
6. Analyze Student Performance
7. Class Analysis
8. Generate Student Report
9. Exit

Enter your choice: 
```

10.2 Adding a Student

The system allows the user to enter a unique Student ID and student name and stores the student record successfully.



```
Enter your choice: 1
Enter Student ID: 001
Enter Student Name: Rahul
Student added successfully!

STUDENT PERFORMANCE ANALYZER

1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Enter / Update Marks
6. Analyze Student Performance
7. Class Analysis
8. Generate Student Report
9. Exit

Enter your choice: 
```

10.3 Entering Marks

The system allows marks to be entered for CSE, Calculus, English, and EVS. Input validation ensures that marks remain within the range of 0 to 100.

```
Enter your choice: 5
Enter Student ID: 001

Enter marks out of 100:
Enter marks for CSE: 80
Enter marks for Calculus: 67
Enter marks for English: 46
Enter marks for EVS: 39
Marks entered successfully!

STUDENT PERFORMANCE ANALYZER

1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Enter / Update Marks
6. Analyze Student Performance
7. Class Analysis
8. Generate Student Report
```

10.4 Individual Performance Analysis

The system calculates the student's total marks and average, determines the grade, and displays the pass/fail result.

```
Enter your choice: 6
Enter Student ID: 001

PERFORMANCE ANALYSIS
Student ID: 001
Name: Rahul

Subject Marks:
CSE : 80
Calculus : 67
English : 46
EVS : 39

Total: 232
Average: 58.0
Grade: D
Result: Pass

STUDENT PERFORMANCE ANALYZER
```

10.5 Class Analysis

The class analysis feature calculates the class average, identifies the highest and lowest performers, and displays the grade distribution.

```
9. Exit

Enter your choice: 7

CLASS ANALYSIS
Class Average: 60.81

Highest Performer:
ID: 002
Name: Dino
Average: 83.75

Lowest Performer:
ID: 1
Name: a
Average: 18.5

Grade Distribution:
A+ : 0
A : 2
B : 0
C : 0
D : 1
```

10.6 Student Performance Report

The system generates an individual performance report containing subject-wise marks, total marks, average, grade, and result.

```
Enter Student ID for report: 003

STUDENT REPORT

Student ID: 003
Student Name: Franky

Subject Marks:
CSE : 89
Calculus : 84
English : 89
EVS : 70

Total Marks: 332
Average: 83.0
Grade: A
Result: Pass

STUDENT PERFORMANCE ANALYZER
```

10.7 Results

The implemented system successfully performs the required student management, marks management, individual performance analysis, class analysis, and report generation operations. The screenshots demonstrate that the application accepts user input, validates marks, performs the required calculations, and displays the resulting information through the menu-driven interface.

11. Testing

Testing was performed to verify that the Student Performance Analyzer functions correctly for normal inputs, invalid inputs, and different student records. The main features of the system were tested individually to ensure that the expected outputs were produced.

11.1 Functional Testing

The following test cases were used to verify the major functions of the system:

Test Case	Input/Operation	Expected Result	Status
Add Student	Enter a new Student ID and name	Student is added successfully	Pass
Duplicate Student ID	Enter an existing Student ID	Duplicate ID is rejected	Pass
Enter Marks	Enter marks from 0 to 100	Marks are stored successfully	Pass
Invalid Marks	Enter marks outside 0 to 100	Invalid marks are rejected	Pass
Search Student	Enter an existing Student ID	Student record is displayed	Pass
Invalid Student ID	Enter a non-existing Student ID	Student not found message is displayed	Pass
Performance Analysis	Analyze a student with marks	Total, average, grade, and result are displayed	Pass
Class Analysis	Analyze multiple student records	Class statistics are displayed	Pass
Generate Report	Enter an existing Student ID	Student performance report is generated	Pass

11.2 Testing Result

The tested features produced the expected results. Input validation, student record handling, marks processing, performance calculations, class analysis, and report generation were verified through the test cases.

12. Challenges

During the development of the Student Performance Analyzer, several challenges were encountered and addressed:

12.1 Managing Multiple Modules

Organizing the project into separate Python modules required careful separation of responsibilities and correct use of imports between files.

12.2 Input Validation

Handling invalid user input, especially non-numeric marks and marks outside the range of 0 to 100, required additional validation and error handling.

12.3 Maintaining Student Records

Ensuring that Student IDs remain unique and that students can be searched and updated correctly required proper record handling.

12.4 Designing Class Analysis

Calculating class-level statistics such as the average, highest performer, lowest performer, and grade distribution required processing the marks of multiple students correctly.

12.5 Debugging Module Integration

While connecting the different modules through the main program, issues such as incorrect imports and function connections had to be identified and corrected.

13. Learnings

The development of the Student Performance Analyzer provided practical experience in applying programming and problem-solving concepts to a complete project.

13.1 Problem-Solving Skills

The project improved the ability to break a larger problem into smaller tasks and design a logical solution before implementation.

13.2 Python Programming

The project provided practical experience with Python variables, data types, conditional statements, loops, functions, lists, dictionaries, input/output operations, and error handling.

13.3 Modular Programming

Working with separate Python modules demonstrated how a program can be divided into smaller components based on their responsibilities and connected using imports.

13.4 Data Validation

The project demonstrated the importance of validating user input to prevent invalid data from affecting program execution and calculations.

13.5 Testing and Debugging

Testing different operations and handling errors helped develop practical debugging skills and showed the importance of verifying program behavior before considering the project complete.

13.6 Documentation and Project Organization

Preparing architecture diagrams, workflow diagrams, UML diagrams, screenshots, and project documentation provided experience in presenting a software project in a structured manner.

14. Future Enhancements

The current version of the Student Performance Analyzer is designed as a basic console-based application. The following enhancements could be considered in future versions:

14.1 Persistent Data Storage

Student records could be stored in files or a database so that data is preserved even after the program is closed.

14.2 Graphical User Interface

A graphical user interface could be added to make the application more interactive and easier to use.

14.3 More Subjects

The system could be extended to support additional subjects instead of using only the current four predefined subjects.

14.4 Advanced Performance Analysis

Future versions could include charts, performance trends, subject-wise comparisons, and additional statistical analysis.

14.5 User Authentication

Login functionality could be introduced to provide different access levels for students and teachers.

14.6 Exportable Reports

Reports could be exported automatically into formats such as PDF or CSV for easier sharing and record keeping.

15. References

1.VITYathi:<https://vityarthi.com/>

2.Course Material